T13 Chests

CRE132 worksheet

What you'll make

Treasure chests. Walk up to one and press E: the lid opens, a sound plays, and the item's icon floats up out of the chest and fades while the item lands in your inventory. One chest holds the gold key, which opens a pen with a gold chest inside.

Then the chests **remember**. Stop the game and press Play again: the chests you opened are still open and empty, and your inventory still has what was in them. That's S08's save file again, this time saving lists of chest ids and item names.



Figure 1: The finished scene: three iron chests in the meadow and a gold chest in a locked pen.

Before you start

1. Unzip `CRE132_T13_Chests_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T13_Chests_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T13_Chests`.
3. Press **Play**. You can walk (WASD or arrows) and attack (J). The chests are solid but don't open, and the pen's **Locked Door** (T10) needs a gold key you can't get yet.

Under **Chests** in the Hierarchy are **Chest 1**, **Chest 2**, **Chest 3** and **Gold Chest**. Each already has an **Animator**, a solid **Box Collider 2D**, a **Circle Collider 2D** trigger (radius 1.3) that marks how close you need to be, and a hidden child called **Prompt** (the "E"). There's also an empty object called **Treasure Save**.

> **The chest animation is already made.** Mystic Woods' `chest_01` (iron) and `chest_02` (gold) are 64 × 16 strips, sliced into four 16 × 16 frames. Each Animator Controller (`Animation/Controllers/`

`Chest_Iron` and `Chest_Gold`) has a **Closed** state (frame 0) and an **Opening** state (all four frames, played once), joined by a transition on an **Open** trigger. Double-click one to look.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

You'll write three scripts. **Chest** uses the other two, so make them first.

1. What you got: ItemReveal

Make a folder `Assets/_game/Scripts/T13_Chests` and create `ItemReveal` in it:

```
// The picture of an item rising out of a chest and fading away, so you can see what you
// got. The chest makes one and calls Show; it removes itself when it's done.
using System.Collections;
using UnityEngine;

public class ItemReveal : MonoBehaviour
{
    [SerializeField] private float riseDistance = 1f;
    [SerializeField] private float duration = 1.2f;

    public void Show(Sprite icon)
    {
        GetComponent<SpriteRenderer>().sprite = icon;
        StartCoroutine(Rise());
    }

    private IEnumerator Rise()
    {
        SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
        Vector3 start = transform.position;
        float time = 0f;

        while (time < duration)
        {
            time = time + Time.deltaTime;
            // 0 at the start, 1 at the end.
            float progress = time / duration;
            transform.position = start + new Vector3(0f, riseDistance * progress, 0f);

            // Fully visible for the first half, then fade out over the second half.
            Color colour = spriteRenderer.color;
            colour.a = Mathf.Clamp01(2f - 2f * progress);
            spriteRenderer.color = colour;
            yield return null;
        }

        Destroy(gameObject);
    }
}
```

- **A coroutine** (`IEnumerator`, `yield return null`) runs a little each frame. The `while` loop moves the icon up and fades it over `duration` seconds, then the object destroys itself.
- **`progress`** goes from 0 to 1. Multiplying by `riseDistance` moves the icon from the chest to 1 unit above it.

- **`2f - 2f * progress`** is 2 at the start and 0 at the end. **`Mathf.Clamp01`** keeps it between 0 and 1, so the alpha stays at 1 for the first half, then falls to 0.

Make the prefab: choose **GameObject > Create Empty** and name it **Item Reveal**. Add a **Sprite Renderer** with **Sorting Layer Foreground** (so it draws over the chest), and add **ItemReveal** (leave **Rise Distance** 1 and **Duration** 1.2). Make a folder `Assets/_game/Prefabs/Effects`, drag **Item Reveal** into it, then delete it from the Hierarchy. It has no sprite of its own: `Show` gives it one.

2. Remembering: TreasureSave

Create `TreasureSave` in the same folder:

```
// Remembers which chests have been opened and what's in the player's inventory, in a
// small JSON save file (the S08 way). Opened chests stay open, and their loot stays
// yours, after you stop and press Play again. To start afresh, right-click this
// component's title in the Inspector > Delete Save.
using System.Collections.Generic;
using System.IO;
using UnityEngine;

public class TreasureSave : MonoBehaviour
{
    // What goes in the file. A class inside a class: only TreasureSave needs it.
    [System.Serializable]
    private class TreasureData
    {
        public List<string> openedChests = new List<string>();
        public List<string> items = new List<string>();
    }

    [SerializeField] private Inventory playerInventory;
    // Every item that can be saved, so a saved name can be turned back into its ItemData
    // asset.
    [SerializeField] private ItemData[] allItems;

    private TreasureData data = new TreasureData();

    private string SavePath
    {
        get { return Path.Combine(Application.persistentDataPath, "treasure.json"); }
    }

    // Awake, so the save is loaded before any chest's Start asks about it.
    private void Awake()
    {
        if (File.Exists(SavePath))
        {
            data = JsonUtility.FromJson<TreasureData>(File.ReadAllText(SavePath));
        }
    }

    // Start, so the Inventory exists. Put the saved items back by name.
    private void Start()
    {
        foreach (string itemName in data.items)
        {
            ItemData item = FindItem(itemName);
            if (item != null)
            {
```

```
                playerInventory.Add(item);
            }
        }
    }

    public bool IsOpened(string chestId)
    {
        return data.openedChests.Contains(chestId);
    }

    // Called by a chest when it opens (after it has added its item), and saves straight
    // away.
    public void MarkOpened(string chestId)
    {
        data.openedChests.Add(chestId);

        data.items.Clear();
        foreach (ItemData item in playerInventory.Items)
        {
            data.items.Add(item.itemName);
        }

        File.WriteAllText(SavePath, JsonUtility.ToJson(data, true));
        Debug.Log("Saved to " + SavePath);
    }

    private ItemData FindItem(string itemName)
    {
        foreach (ItemData item in allItems)
        {
            if (item.itemName == itemName)
            {
                return item;
            }
        }
        return null;
    }

    // [ContextMenu] adds this to the component's right-click menu in the Inspector, even
    // outside Play.
    [ContextMenu("Delete Save")]
    private void DeleteSave()
    {
        File.Delete(SavePath);
        Debug.Log("Deleted " + SavePath);
    }
}
```

- **Save what you can't rebuild from the scene:** which chests are open, and what the player is carrying. `TreasureData` holds two lists of text, and **`JsonUtility`** turns it into JSON and back (S08). It's marked **`[System.Serializable]`** so JsonUtility can see inside it.
- **Loading order matters.** The file is read in **`Awake`**, so it's ready before any chest's `Start` asks `IsOpened`. The items go back into the inventory in **`Start`**, once every object exists.
- **The file stores item names, not items.** A name is just text, so `FindItem` looks through **All Items** for the asset with that name.
- **`[ContextMenu]`** adds **Delete Save** to the component's right-click menu, so you can start afresh.

Add it to the **Treasure Save** object. Drag the **Player** into **Player Inventory**. Set the size of **All Items** to **4** and drag in the **Coin**, **Blue Gem**, **Gold Key** and **Potion** assets from `Assets/_game/ScriptableObjects/Items/`.

3. The chest itself: Chest

Create `Chest` in the same folder. It's the longest script, but most of it is patterns you've met: reach and the E prompt work like T11's Talkable, and the player is "the thing with an Inventory", as in T09.

```
// A chest the player opens with Interact (E). It plays its opening animation and sound,
// shows the item rising out of it, puts the item in the player's Inventory (T09), and
// tells the TreasureSave that it's open, so it stays open (and empty) the next time you
// play.
using UnityEngine;
using UnityEngine.InputSystem;

public class Chest : MonoBehaviour
{
    // Every chest needs a different id: the save file remembers opened chests by it (like
    // S08's coin names).
    [SerializeField] private string chestId = "Chest 1";
    [SerializeField] private ItemData contents;
    [SerializeField] private int amount = 1;

    [SerializeField] private InputActionReference interactAction;
    [SerializeField] private TreasureSave treasureSave;
    // The "E" shown while the player can open it.
    [SerializeField] private GameObject prompt;
    // The item picture that rises out of the chest.
    [SerializeField] private ItemReveal revealPrefab;
    [SerializeField] private AudioClip openSound;

    private Animator animator;
    private Inventory playerInventory;
    private bool playerInReach;
    private bool isOpen;

    private void Awake()
    {
        animator = GetComponent<Animator>();
        prompt.SetActive(false);
    }

    private void Start()
    {
        // Opened in an earlier game: jump straight to the last frame of the opening
        // animation. (Play's third number is how far through the animation to start:
        // 1 = the end.)
        if (treasureSave.IsOpened(chestId))
        {
            isOpen = true;
            animator.Play("Opening", 0, 1f);
        }
    }

    private void OnEnable()
    {
        interactAction.action.Enable();
    }
```

```
    private void OnDisable()
    {
        interactAction.action.Disable();
    }

    private void Update()
    {
        prompt.SetActive(playerInReach && !isOpen);

        if (playerInReach && !isOpen && interactAction.action.WasPressedThisFrame())
        {
            Open();
        }
    }

    private void Open()
    {
        isOpen = true;
        animator.SetTrigger("Open");
        if (openSound != null && AudioManager.Instance != null)
        {
            AudioManager.Instance.PlaySound(openSound);
        }

        ItemReveal reveal = Instantiate(revealPrefab,
            transform.position + new Vector3(0f, 0.7f, 0f), Quaternion.identity);
        reveal.Show(contents.icon);

        for (int i = 0; i < amount; i++)
        {
            playerInventory.Add(contents);
        }
        treasureSave.MarkOpened(chestId);
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        // The player is the thing with an Inventory.
        Inventory inventory = other.GetComponent<Inventory>();
        if (inventory != null)
        {
            playerInventory = inventory;
            playerInReach = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.GetComponent<Inventory>() != null)
        {
            playerInReach = false;
        }
    }
}
```

- **`chestId`** must be different for every chest: the save file remembers opened chests by it, just as S08's coins needed unique names.

- **`Open`** does everything at once: `SetTrigger("Open")` starts the animation, the **AudioManager** (S05) plays the sound, an **ItemReveal** appears 0.7 units above the chest, the items go in the inventory, and the TreasureSave is told.
- **`animator.Play("Opening", 0, 1f)`** jumps straight to the **Opening** state on layer 0, at 1 (100 %) of the way through. A chest opened in an earlier game shows its open lid without animating.
- **`prompt.SetActive(...)`** in `Update` shows the "E" only while you're in reach and the chest is still shut.

Add it to all four chests (select all four, Add Component), then fill in the slots that are the same on every chest: **Interact Action Player/Interact**, **Treasure Save** the **Treasure Save** object, **Reveal Prefab** your **Item Reveal** prefab, and **Open Sound** `GS2_Treasure_Chest_Open` (`Assets/_game/Audio/SFX/`). Then select each chest on its own and set the rest, dragging its own child **Prompt** into **Prompt**:

Chest	Chest Id	Contents	Amount
Chest 1	Chest 1	Coin	5
Chest 2	Chest 2	Gold Key	1
Chest 3	Chest 3	Potion	1
Gold Chest	Gold Chest	Blue Gem	3

Test it: press Play, walk up to **Chest 1**: the **E** appears. Press **E**: the lid opens, a coin rises and fades, and the inventory shows 5 coins. Open **Chest 2**, take the gold key to the pen's door and open the **Gold Chest**. The Console shows `Saved to ...` with the file's path each time.

Now stop and press **Play** again. The chests you opened are open and empty, and the coins, key and gems are back in your inventory. To start afresh, stop the game, right-click the **Treasure Save** component's title and choose **Delete Save**.

Check your work

- ☐ The E prompt appears only near a closed chest.
- ☐ Each chest opens once, plays its animation and sound, shows its item rising, and adds the right number to the inventory.
- ☐ The gold key from Chest 2 opens the pen.
- ☐ After stopping and playing again, opened chests stay open and your inventory is refilled.
- ☐ **Delete Save** makes every chest closed and full again.
- ☐ Compare with `CRE132_T13_Chests_Finished` if something behaves differently.

Key points

- **Every saved thing needs a unique id**, for the same reason S08's coins needed unique names.
- **Save what you can't rebuild from the scene:** which chests are open, and what the player is carrying. Items are saved by name and looked up again when loading.
- **`Animator.Play(state, layer, normalizedTime)`** jumps straight to any point of an animation: 0 is the start, 1 the end.

If something goes wrong

- **NullReferenceException in Chest's Awake:** the **Prompt** slot is empty.
- **NullReferenceException in Chest's Start:** the **Treasure Save** slot is empty.
- **NullReferenceException when you press E: Reveal Prefab** or **Contents** is empty.
- **Two chests open together after a restart:** they have the same **Chest Id**.
- **The chest opens but the lid doesn't move:** the Animator's trigger must be called `Open` exactly, and the state `Opening`.

- **After a restart, items vanish from the inventory:** that item isn't in **All Items**, so its name can't be turned back into an asset.
- **No icon rises:** the Item Reveal's Sprite Renderer is on a sorting layer behind the chest; use **Foreground**.

Try it

1. Play a second, short sound as the item appears, through the AudioManager.
2. A chest that needs the gold key to open: check the inventory in `Open`.
3. A random chest: pick its contents from a list when it's opened.
4. Spot the trap: open the pen with the key, open **Chest 3** (which saves), then stop and play again before opening the Gold Chest. The pen is locked again and the key is gone. Fix it by remembering opened doors too: a `doorId` on LockedDoor and a third list in `TreasureData`. (T14 avoids the trap by turning saving off.)